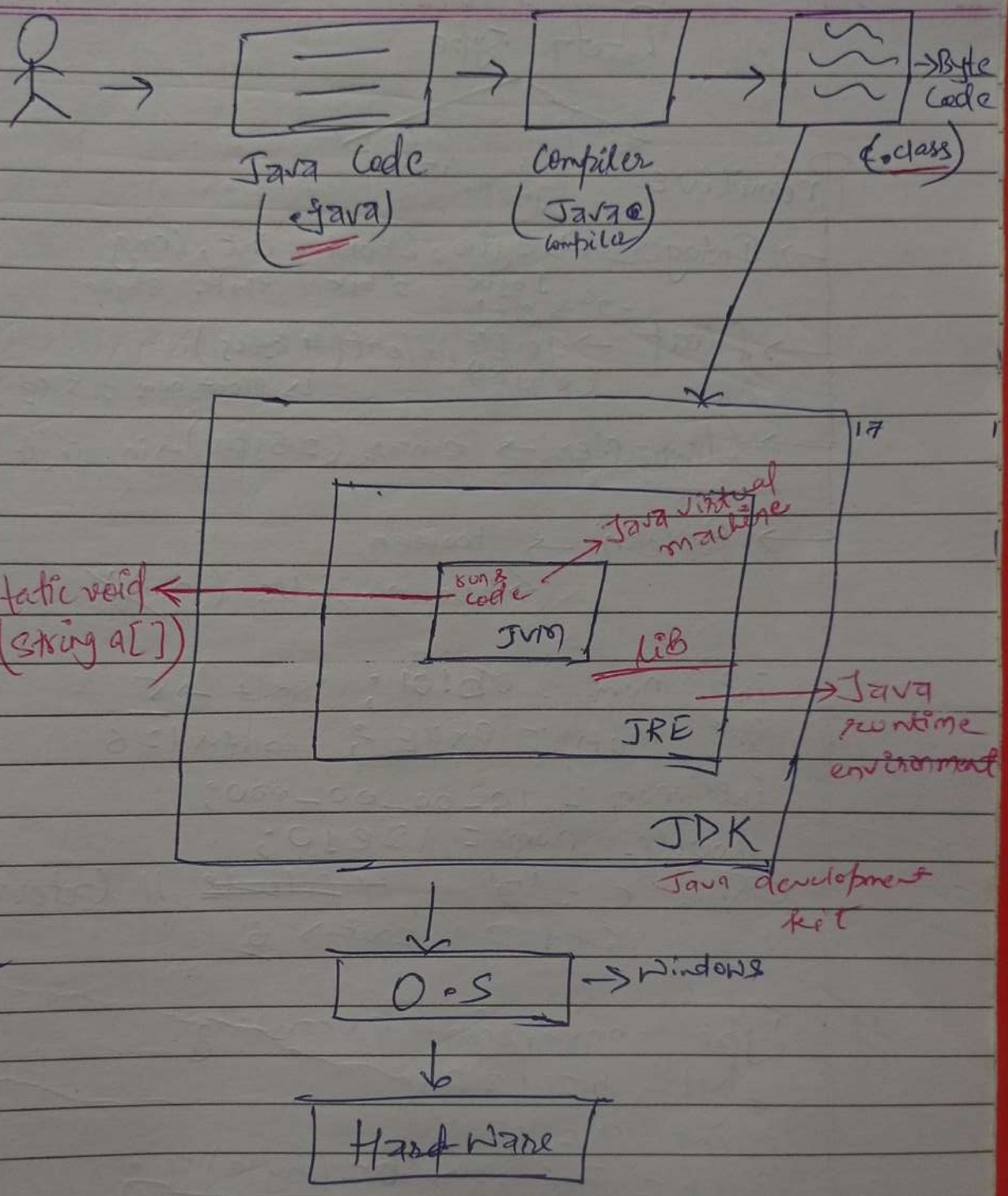
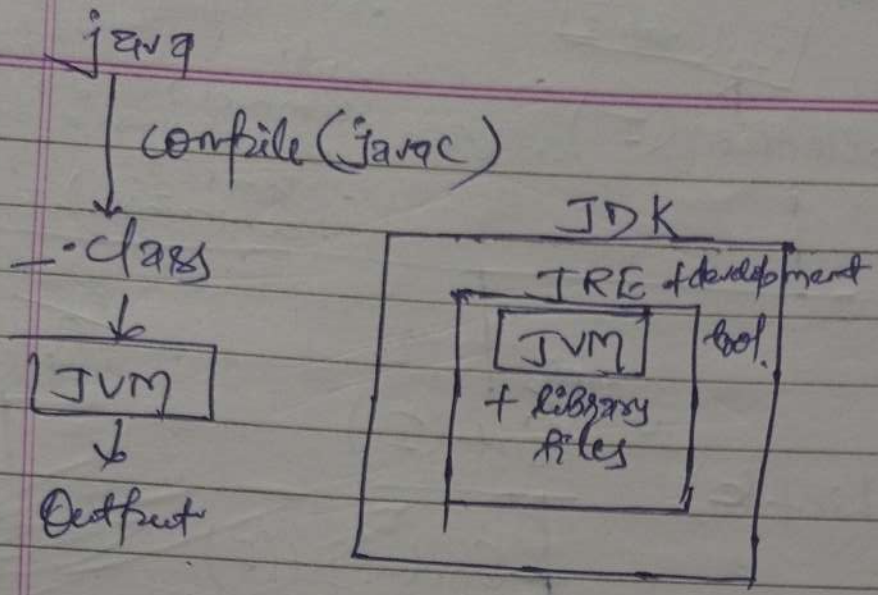
JAVA

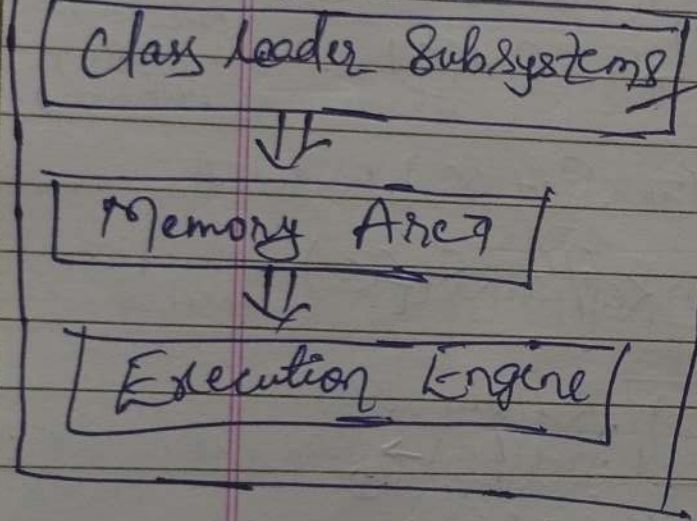
javac code.java
java code (class)

Page No.:
Date: 8/7/24





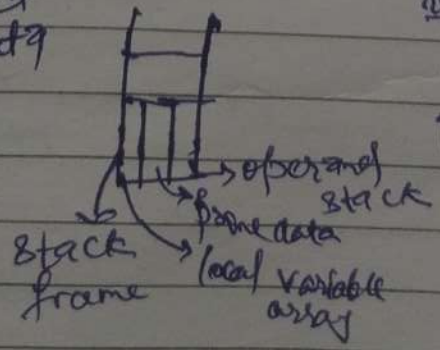
* JVM *



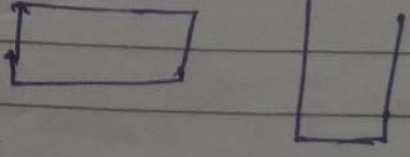
- Loading
  1. Boot strap class loader
  2. Extension class loader
  3. Application class ..
- Linking
  - → Verify
  - → Resolve
  - → Prepare
- Initialization

Memory Area

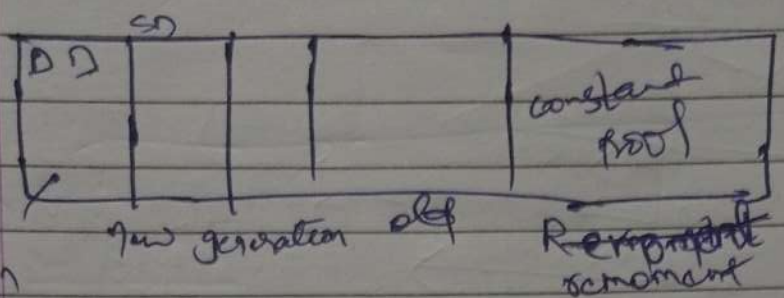
- Method area → Class data like methods
- Heap area → object data
- Stack area



- PC Register
- Native method stack



Heap



Execution Engine

Interpreter

JIT Compiler

Intermediate code generator
↓
Code optimizer
↓
target code generator
↓
m/code

Garbage Collector

1. Serial
2. Parallel
3. Concurrent mark & sweep (CMS)

Modifiers

- Access
  - scope
    - default → package
    - public
    - protected
    - private
- Non Access
  1. static (class level)
     - variable
     - method → override ×
     - class → can't inherit
  2. final
  3. Synchronized
  4. abstract
  5. volatile
  6. Native
  7. Transient

Exception Handling

Throwable → Root class

Exception: Description
StackTrace

Throwable
- Exception
  - Runtime Exception
    1. IndexOutOfBoundException
       - ArrayIndexOutOfBoundEx
       - StringIndexOutOfBoundException
    2. NullPointerException
    3. ArithmeticException
  - IOException
    - EOF Exception
    - File not found
- Error
  - Virtual Memory Error
    - StackOverflow
    - Out of Memory Error

Page No.:
Date: / /

Data type

Primitive

- Integer → byte, short, int, long
  - byte: 1 byte, -2^8 to 2^8-1, i.e. -128 to 127
  - short: 2 byte
  - int: 4 byte
  - long: 8 byte
- float → double, float (4 bytes.)
  - (8 bytes)
  - float num = 5.6f; // ? double
- Character → char (2 Bytes) → in single quote
- Boolean → boolean
  - true or false (0, 1)

```
int num = 0b101;   sout → 5
int num = 0x7E;    sout → 126
int num = 10_00_00_000;
double num = 12e10;
char c = 'a'   // literals
    c++;   sout → b
```

Type conversion & casting.

```
byte b = 127
int a = 12;
b = a;   → b = (byte) a;
a = b;
```

conversion — casting

```
float f = 5.6f;
int i = (int) f;
```

```
byte b = 127;
int a = 257;
b = a;   → b = (byte) a;
257 % 256 = 1
```

System.out.print();
System.out.println();

& ← AND | — OR ! — NOT

short circuit && ← || ←

Calculator calc = new Calculator();
object creation
reference variable

import java.util.Scanner; (or *) → for all

Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int arr[] = new int[n];
int [][] arr = new int[4][4];

enhanced for loop

for (type : arr) {
for (int a : arr) { sout(a); }
}

* int nums[] = {5, 6, 7};
* int nums[] = new int[4]; ← dynamic, all values are 0
* int random_no. = (int)Math.random()*100;
0.00 - 1.00

for(int n[] : nums)
{ for (int m : n)
{ sout(m+" "); } soutln(); }

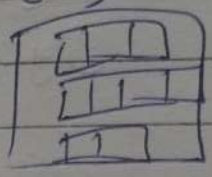
nums[][]

Page No.:
Date: / /

Jagged array

```
int nums[][] = new int[3][];
nums[0] = new int[3];
nums[1] = new int[4];
nums[2] = new int[2];
```

for length → nums[i].length



* String name = new String(); "abc";

String is a class in java so each String declared object is created

→ name.charAt(1); .replace('x','y');
→ name.concat("x")
.indexOf(); lastIndexOf()

→ n.split(" ") arr
→ n.join()
→ .toUpperCase()
→ .toLowerCase()

StringBuffer / StringBuilder → Mutable String → change
→ Immutable String → unchange

```
StringBuffer sb = new StringBuffer();
sb.capacity(); → 16;
sb.length();
sb.append("title");
sb.deleteCharAt();
String str = sb.toString();
sb.insert(0, "x"); (index, value)
x.equals(sb);
sb.insert(offset:_; str:"_")
sb.setLength();
sb.ensureCapacity(min cap:100);
.setCharAt(index, value);
→ sb.equalsIgnoreCase(sb.reverse().toString())
.substring(_,_)
```

("name") 16+4 4

name title

Char arr[] = str.toCharArray();

Character.toUpperCase(arr[i]);

return new String(arr)

Character.isDigit()

n >> k	n << k	XOR	1010
n / 2^k	n * 2^k	diff-1	0011
		same-0	1001

Set ith bit → n | (1<<i)

011011 ⇒ 011111

↳ 000100

000000

onset → 0

0|0|00 ⇒ 0|0|00

↑

1000

class Mobile {

String brand

int price;

static String name;

public Mobile() { const }

can only access static variables

static { name = "Phone"; }

public static void show() { sout("static"); }

static methods

Pcotx class Demo

{ public static void main(String a[])

{ Mobile obj = new Mobile();

Class.forName(className: "Mobile");

loads class without creating object

Mobile.name = "Phone", can define → class static variable

Mobile.show(); can call static method directly with class Pink.

→ every class in Java extends the object class

Page No.:
Date: / /

```
Class A {
    public A() { Sout("A") }

    Public A(int n) { }

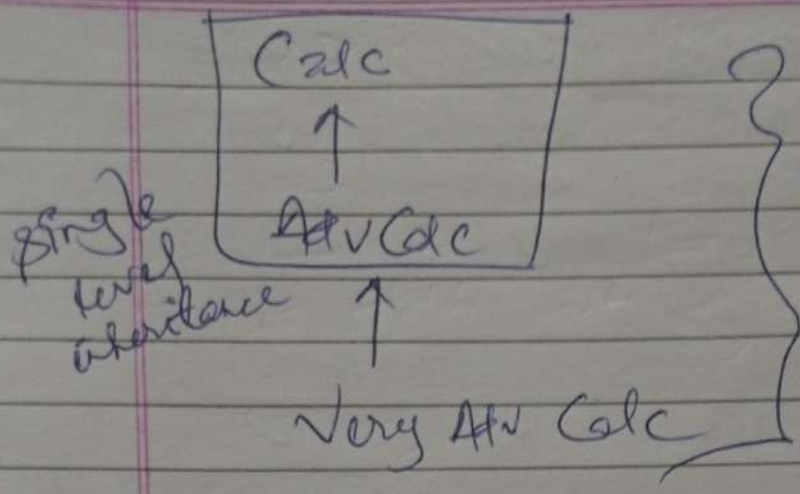
Class B extends A {
    Public B() { super(); → call constr. of super class
                 Sout("--"); }

    Public B(int n)
    { super(n); → can make it Parameterised
      this(); → it execute constructor of same class

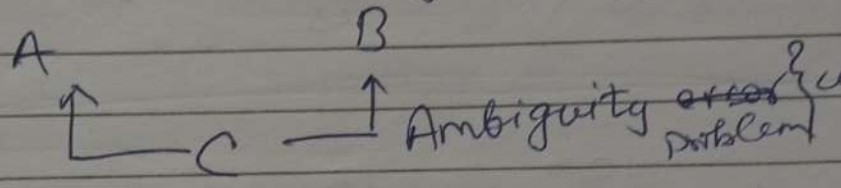
Public class Demo
{ public static void main(String a[])
    { B obj = new B();
      new A(); anonymous object
    }
}
```

Camel casing

class & interface – Calc, Runable
variable and method – marks, sRaw()
constants – PIE, BRAND
showMyMarks()



- Multiple inheritance in java will not work.



- Java overwrites the methods of the parent class if method name is same in child & parent class.

import tools.Calc*; → to import all files/class of that folder
folder class
import java.util.ArrayList;
folder folder
ArrayList list = new ArrayList();

→ Whichever inbuilt class we use in java belongs to a package.

→ package other.tools;
public class Calc
{

}

	Private	Protected	Public	Default
Same Class	Yes	Yes	✓	✓
Same Package subclass	no	Yes	✓	✓
Same package non-subclass	no	Yes	✓	✓
diff. package subclass	no	Yes	✓	X
diff. package non-subclass	no	no	✓	X

Polymorphism

behaviour

many

- Compile time (early binding)
  - Overloading
    - add(int, int)
    - add(int, int, int)
- Run time (late binding)
  - Overridding

```
A { add(int, int)
  }
B { add(int, int)
  }
```

Dynamic Dispatch

public ... toString( ) ...

Page No.:
Date: / /

```
Class A {
    public void show() { sout("In A show"); show1() }
}
Class B extends A
{ public void show() { sout("In B show"); show2() }

public class Demo
{ public static void main(String a[])
  { A obj = new A(); / A obj = new B();
    obj.show()  → in A show

    obj = new B();
    obj.show();  → in B show
```

- Upcasting

```
A obj = (A) new B();
obj.show1();
```

- downcasting

```
B obj1 = (B) obj;
obj.show2();
```

→ only works when we have inheritance

final ← variable, method, class → If final keyword is used in class then another class can't extend it.

const variable that can't be changed once assigned.

no one can override method

```
public final void fn();
```

```
Laptop obj = new Laptop()
sout(obj) / (obj.toString());
```

Laptop@7ad041f3 ← (obj)

returns getClass().getName() + "@" + Integer.toHexString(hashCode());

Abstract

Page No.:
Date: / /

```
abstract class Car
{
  public abstract void drive();

  Public void playMusic(){ }
}

Class Wagon extends Car  //Concrete class
{
  public void drive()
  {
  }
}
```

if u extend abstract class then to have it define it.

- U can't create object of abstract class.

```
{ Car obj = new Wagon();
  obj.drive();
  obj.playMusic();
```

Class inside class

```
Class A{ public void show(){
           Class B{ public void config()
                    { Sout("config");
                    }
         }
}

class Demo
{  main(){ A obj = new A();
           obj.show();
           A.B objj = obj.new B();
           objj.show() & config();
```

A.B objj = new A.B();
If class is static:

Page No.:
Date: / /

Anonymous Class

```
Class A { void show() { sout("in A show"); }

public class Demo {
    main(String a[])
    {
        A obj = new A() {
            public void show()
            { sout("in new show"); }
        };
        obj.show();
    }
}
```

its an object of anonymous inner class so we can create object of abstract class too.

Interface

abstract class

```
interface A {
    int age=4;
    void show();
    void config();
}
```

→ public abstract

all variables in interface are final & static

```
class B implements A
{
    public void show() { }
    public void config() { }
}

main() { A obj;
    obj = new B();
    obj.show();
    sout(A.age);
```

- A class can implement multiple interfaces not abstract class.

@Override

Page No.:
Date: / /

Class - class → extends
class - interface → implements
interface - interface → extends

Enumeration (enum)

```
enum Status {
    Running, Failed, Pending, Success;
}

main () {
    Status s = Status.Running;
    sout (s); → Running;
    sout (s.ordinal()); → 0 (order)
    Status [] ss = Status.values();
                ↳ it will give an array.
```

---

objects in same class

```
enum Laptop {
    Macbook (2000), XPS (2200), Surface (), Lenovo (8700)
    private int price;
    private Laptop() { price = 500; }
    private Laptop (int price) { this.price = price;
                                 sout (this.name()); }
    public int getPrice() { return price; }
    public void setPrice (int price) { this.price = price; }
```

@functional Interface

```
interface A
{ void show(); }

main()
{
    A obj = () -> System.out.println("in show");
    obj.show();
}
```

lambda exp

lambda expression

only work with functional interface

anonymous class

```
main()
{
    A obj = new A()
    { public void show()
      { Sout("in show"); }
    };
    obj.show();
}
```

```
main()
{
    A obj = i -> Sout("in show"+i);
    obj.show(5);
}
```

Types of Interface.

Normal	Functional / SAM	Marker
interface Abc	interface Abc	interface Abc
{ 2 or more than 2 methods }	{ 1 method }	{ no methods }

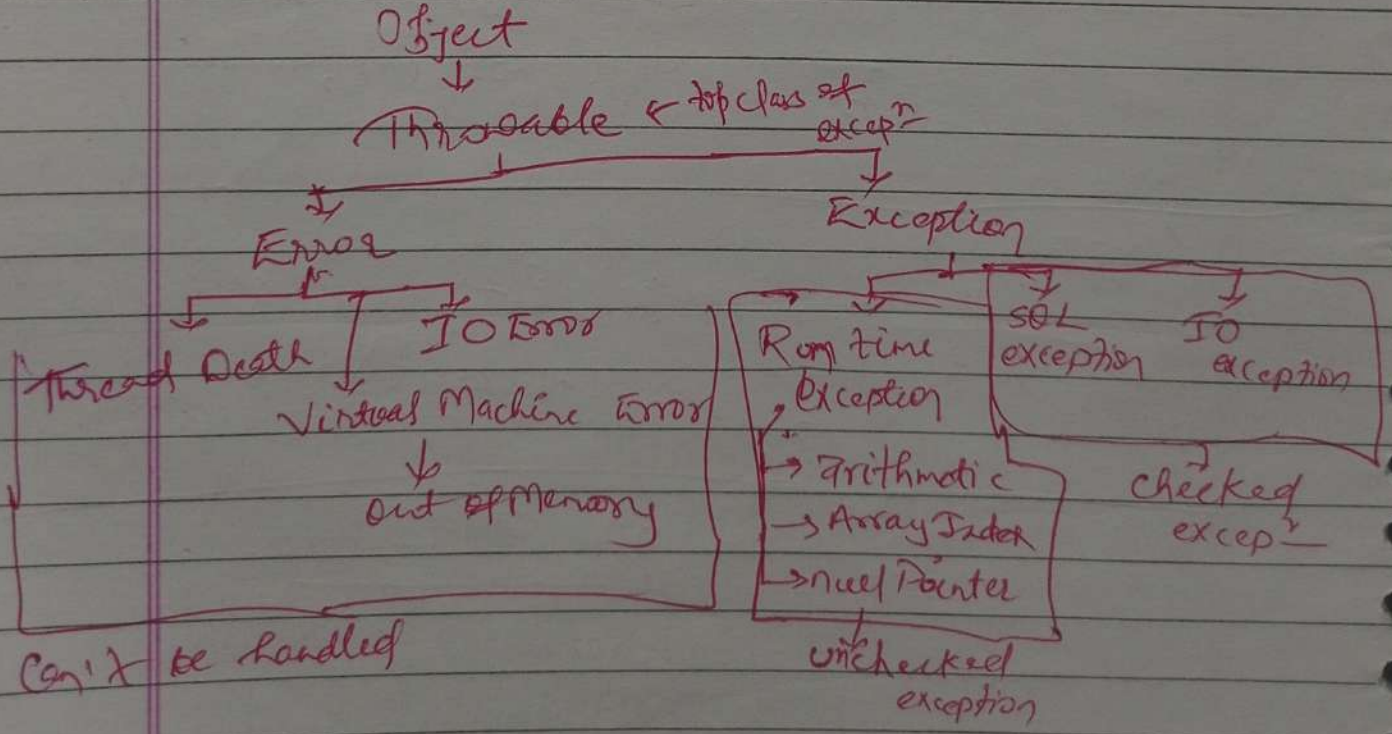
Exceptions

Error:

1) Compile time error → Spelling mistake / Syntax
2) Logical error → logic is wrong or output
3) Runtime error → exception handling

Exception Handling

```
main() { int i=0, j=0;
try { j = 18/i; }
catch (Exception e) obj
{ sout("something went wrong"); }
sout (j);
```



```
main(){ int i=20, j=0;
    try { j=18/i;
          if(j==0)
throwing exception ← throw new ArithmeticException("I want to print");
          throw new myException("I don't want to print 0"); //
    }
catch(ArithmeticException e){
          j=18/i;
          Sout("thats default output"+e);
}
Catch(myException e){
}
Catch(Exception e) ← default exception
          Sout("Something went wrong"+e);
}
```

```
Class myException extends Exception
{ public myException(String str)
    { super(str); }
}
```

Throws

Page No.:
Date:

if error occurs it is thrown to parents class

```
class A {
    public void show() throws ClassNotFoundException
    {
        Class.forName(className: "Calc");
    }
}

Public class Demo.
    static { sout(" .Class Loaded"); }

    main() throws ClassNotFoundException
    {
        A obj = new A();
        obj.show();
        try { obj.show(); }
        } catch (ClassNotFoundException e)
            { e.printStackTrace(); }
```

↑ prints entire stack

```
try {

}
catch {

}
finally { runs irrespective of try & catch
          used to close resources.
```

Page No.:
Date: / /

* Taking Inputs from user.

```
InputStreamReader in = new InputStream-
                       Reader(System.in);
BufferReader br = new BufferedReader(in);

num = Integer.parseInt(br.readLine());

br.close();
```

```
Scanner sc = new Scanner(System.in)
num = sc.nextInt();
```

Threads.

```
Class A extends Thread
{ public void run()
    { for(int i=1; i<=100; i++)
        { System.out("hi");
          try catch Thread.sleep(millis:10);
        }
}
Class B extends Thread
{ public void run()
    for(int i=1; i<=100; i++)
        { System.out("Hello");
          try{ Thread.sleep(millis:10) }catch{ }
        }
}
main(){ A obj1 = new A();
        B obj2 = new B();
```

it prints simultaneously → obj1.start(); obj2.start();

```
obj1.getPriority();
obj2.setPriority(Thread.MAX_PRIORITY); //(1-10)
```

Page No.:
Date: / /

Class A implements Runnable.

{

}

main() {

Runnable obj1 = new A();

Runnable obj2 = new B();

Thread t1 = new Thread(obj1);

Thread t2 = new Thread(obj2);

t1.start();

t2.start();

Runnable obj1 = () ->

{ for(int i=1; i<=5; i++)

{ sout("hi");

try { Thread.sleep(millis:10);

} catch(InterruptedException e)

{ e.printStackTrace(); }

}

};

Class Counter2

{ int count

public synchronized void increment()

{ count++; }

}

→ it method will be called by only one method at a time

18/2/15

Advanced Array Techniques

1. Sliding Window
2. Kadane's Algo → max sum sub array

2	-6	3	5	4	7

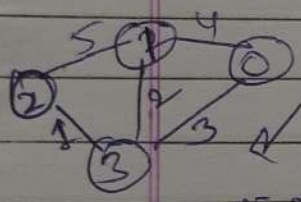
3. Prefix Array
4. Prime Sieve → T: O(n.log log n)
   → S: O(n)

(min spanning tree)

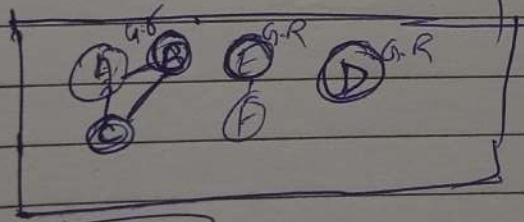
5. Commutable Island → 1. Prim's algo → visited and PQ min key
   Interview bit. → 2. Kruskal algo

Disjoint set

0, 1
1, 2
3, 4
5, 6
4, 5
2, 5



help in finding groups & root



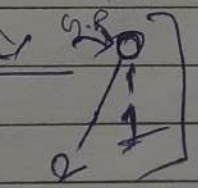
not same group ⇐ A, E

Rank	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

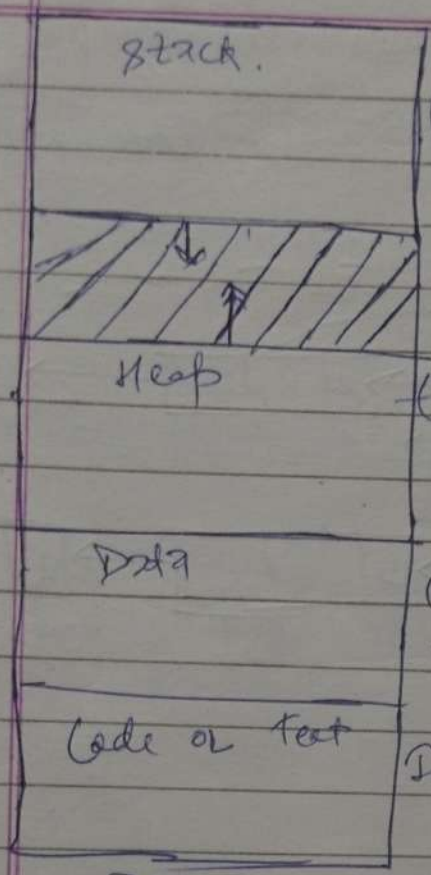
Parent	0	1	2	3	4	5	6
	0	1	2	3	4	5	6

OR itself

OR itself



Right of the group is rank



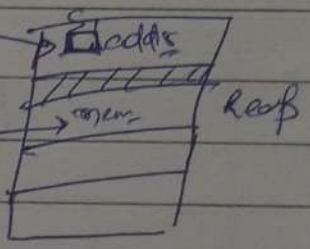
(stack) local variable
function parameters
return address.

(Heap)
dynamic memory alloc ?

Global (Data), Local variable,
variable static

Instruction

Student *S1 = new Student();
? memory is allocated




```
void main(){
int n=5;
int *A = createArray(n);
Print(A[3]);
```

```
int *createArray(int n){
int arr[n]; (static)
for(int i=0;i<n;i++)
arr[i]=i;
return arr;
}
```

101

(int *arr = new int[n];) so dynamically create the array. instead of statically.

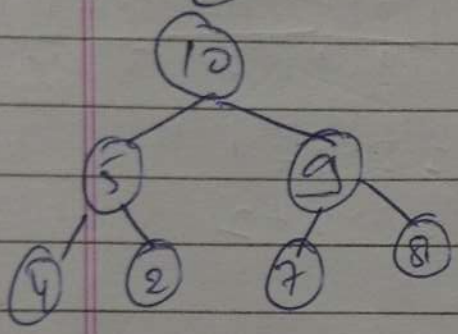


Heap

Max Binary Heap (MaxHeapTree)

Complete binary Tree

[child value < parent value]



10	5	9	4	2	7	8

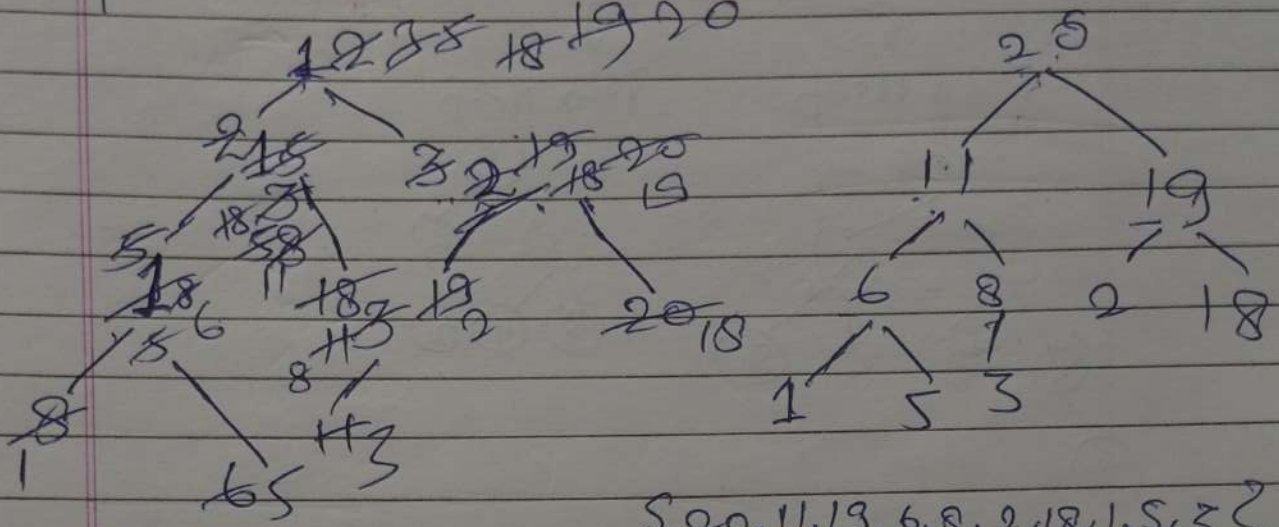
Heap sort

```
for(int i = n/2 - 1 ; i >= 0 ; i--){
        Heapify(i);
}
for(int i = 0; i < n ; i++){
        delete(i);
}
```

K^{th} largest element using MaxHeapTree

connect 4 dots gfg

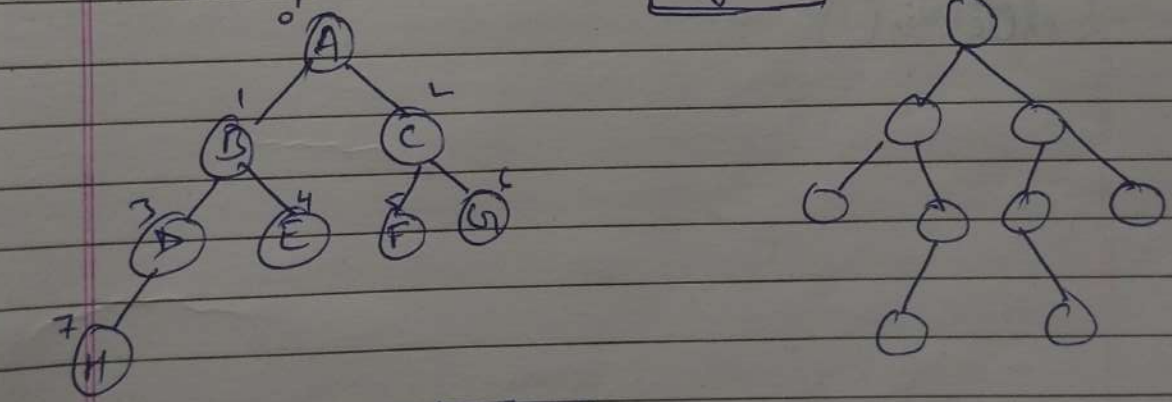
arr = {1, 2, 3, 5, 18, 19, 20, 8, 6, 11}



{20, 11, 13, 6, 8, 2, 18, 1, 5, 3}

0 1 2 3 4 5 6 7 8 9

16/19/24 # Complete BT $\propto \log_2 N$ $\log_2 N$



A	B	C	D	E	F	G	H		
0	1	2	3	4	5	6	7		

i = N/2

i

2i+1

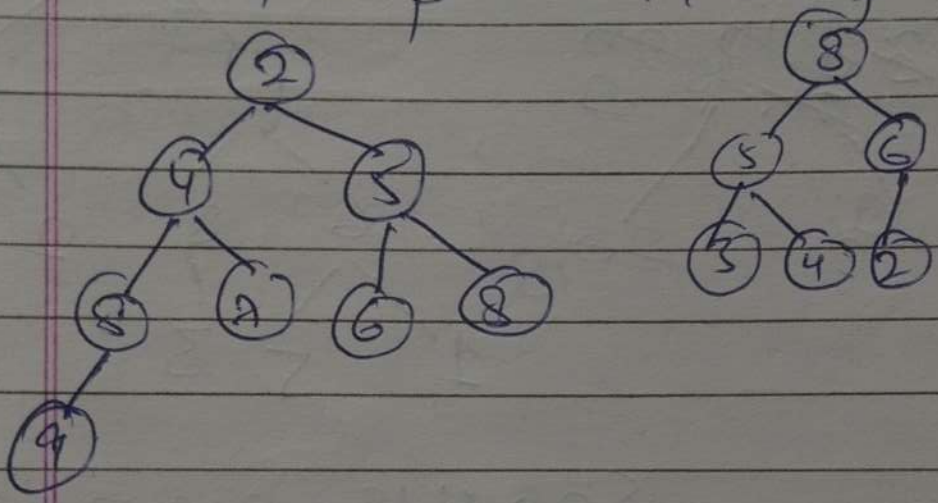
2i+2

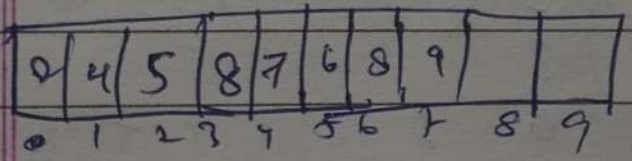
Squed tree

$\propto N$

Page No.:
Date: / /

Binary Heap → Special Complete BT





- GetMin()
- Insert()
- DeleteMin()

Greedy Method

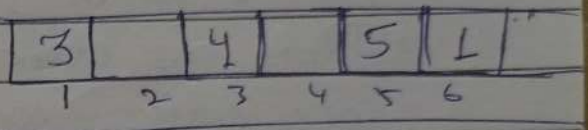
Page No.:
Date 17/10/24

Interval Scheduling

{ }

Job Scheduling

Job id	end-date	Profit
1	6	50
2	5	100
3	1	20
4	3	60
5	5	200 (max)



26/11/24

KMP → Knuth - Morris - Pratt Algorithm

① Create π table of pattern

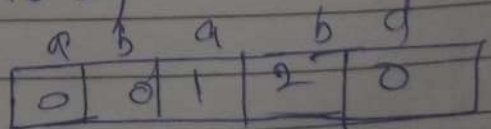
② Start comparing pattern given string
i=1 & j=0 ; start point
compare s[i] & p[i+1]

③ if match i++ , j++
if mismatch
check if j==0 then i++
else set j to corresponding value in π table.

if j == m (matched)
set j=0 & increase counter

m: big of pattern

s: a b a b c a b c a b a b a b d
p: a b a b d



i j
1 0